«Реализация калькулятора на основе Обратной Польской Нотации»

Итоговый проект семестра

Эта лабораторная работа является итоговой работой курса. Будем использоваться **Обратная Польская Нотация (ОПН)**, или *постфиксную нотацию*. Этот метод представления выражений используется в реальных вычислительных системах, компиляторах и интерпретаторах, так как он позволяет производить вычисления без необходимости обработки скобок и учета приоритетов операторов в явном виде.

Ключевой структурой данных для этого проекта является **Стек**. Его применение основа для корректной работы алгоритма конвертации и вычисления.

Структура проекта: Два ключевых модуля

Калькулятор должен состоять из двух последовательно работающих модулей:

1. **Конвертер (Логический модуль):** Принимает на вход строку в стандартной (инфиксной) нотации, например `(3 + 4) * 2`, и преобразует её в строку в ОПН: `3 4 + 2 *`. Требуется реализации **алгоритма сортировочной станции**.
2. **Вычислитель (Арифметический модуль):** Принимает на вход строку, уже представленную в ОПН, и вычисляет конечный результат.

Обязательные технические требования (проверяются при сдаче)

Условия приёмки. Правильный ответ на тестовом примере при нарушении любого пункта ниже **не засчитывается**.

1. **Стек.** Использовать `System.Collections.Generic.Stack<T>` в сдаваемом коде **запрещено**. Обязательна собственная

реализация (`MyStack` или аналогичное имя) поверх вашего `MyArrayList` из Лабораторной работы 5. `Stack<T>` из .NET в справочном материале показан только для понимания принципа работы — в финальном решении его быть не должно.

2. **Токенизация без пробелов.** Функция из Части 1 обязана одинаково корректно обрабатывать выражение как с пробелами (`"3 + 4 * 2"`), так и без них (`"3+4*2"`, `"(3+4)*2"`), и в смешанном виде (`"3+ 4 *2"`). Реализация, которая работает только если между всеми токенами вручную расставлены пробелы, требование не выполняет.
3. `Split` / `Regex` **как основной механизм разбора запрещены в Части 1.** Получать список токенов из ещё не размеченного пробелами выражения через `string.Split(...)` или регулярные выражения — нельзя. Разбор обязан идти посимвольным проходом по строке: индекс `i`, накопление цифр в буфер (`StringBuilder`), посимвольное распознавание операторов и скобок. Вспомогательные методы (`char.IsDigit`, `double.TryParse` для проверки уже собранного токена и т.п.) использовать можно.
   Исключение — Часть 2: там входная строка по условию уже в формате «токены через пробел», и там `Split(' ', StringSplitOptions.RemoveEmptyEntries)` использовать разрешено и ожидается.
4. **Формат чисел.** Токенизатор обязан распознавать целые и вещественные числа с разделителем `.` (например `12`, `3.14`), включая многозначные (`123`, а не только одна цифра за раз). Числа с `,` как разделителем дробной части, с пробелом внутри числа или в экспоненциальной записи (`1e5`) — не поддерживаются и тестами не проверяются.
5. **Унарный минус — вне базовой части.** В Частях 1–4 унарный минус (`-5`, `(-3)`) можно не поддерживать; на таком входе допустимо как некорректное поведение, так и исключение — это не штрафуется. Полноценная поддержка унарного минуса относится к Части 5 (доп. задание) и засчитывается только как осознанно реализованная отдельная логика (с пояснением в коде, как отличили унарный минус от бинарного).
6. **Обязательные сигнатуры.** Минимум следующие публичные методы должны существовать и вызываться именно так:

- `List<string> Tokenize(string expression)` — Часть 1.
- `double Calculate(string rpnInput)` — Часть 2.
- `string ConvertToRpn(string expression)` — Часть 3.
  Сигнатуры можно расширять доп. параметрами со значениями по умолчанию, но базовый вызов с одним строковым аргументом обязан работать без изменений — именно так будет проверяться сдача.

7. **Некорректные символы.** Если во входной строке встречается символ, не являющийся цифрой, точкой, пробелом, оператором (`+ - * / ^`) или скобкой — токенизатор обязан выбросить исключение (`FormatException` или собственное) с понятным сообщением. Молча игнорировать символ или падать с необработанным системным исключением — нельзя.

Задание:

Написать консольное приложение, реализующее калькулятор, работающий с Обратной Польской Нотацией.

Часть 1. Лексический анализатор (Парсер)

Реализуйте метод `List<string> Tokenize(string expression)`, который принимает на вход математическое выражение в виде строки и разбивает его на список токенов (чисел, операторов, скобок) **посимвольным разбором строки** (см. требования 2–3 выше — `Split` / `Regex` по всей строке как основной механизм запрещены).

Метод обязан давать одинаковый результат на обоих вариантах записи одного и того же выражения — это и есть основной тест:

- `"(3 + 4) * 2"` -> `["(", "3", "+", "4", ")", "*", "2"]`
- `"(3+4)*2"` -> `["(", "3", "+", "4", ")", "*", "2"]`

Часть 2. Вычислитель ОПН

Реализуйте функцию `double Calculate(string rpnInput)`.

- Принимает на вход строку в формате ОПН, где токены разделены пробелами (например, `"3 4 2 * 1 5 - 2 ^ / +"`).

- Возвращает результат вычисления в виде `double`.
- Формат входа здесь фиксирован условием (токены всегда через пробел) — использование `string.Split(' ', StringSplitOptions.RemoveEmptyEntries)` для получения списка токенов в этой части **разрешено и ожидается** (см. требование 3).
- Стек для промежуточных вычислений — обязательно ваш `MyStack` (см. требование 1).

Часть 3. Конвертер в ОПН

Реализуйте функцию `string ConvertToRpn(string expression)`.

- Принимает на вход стандартное математическое выражение (с пробелами, без пробелов или в смешанном виде — см. требование 2).
- Возвращает его представление в виде строки ОПН.
- Алгоритм — сортировочная станция Дейкстры; стек операторов — обязательно ваш `MyStack` (см. требование 1).

Часть 4. Сборка и Тестирование (`Main`)

В методе `Main` объедините все компоненты в единую программу:

1. Запросите у пользователя математическое выражение.
2. (Опционально) Разбейте его на токены с помощью парсера.
3. Вызовите **Конвертер** для получения строки в ОПН.
4. Передайте полученную строку в **Вычислитель** для получения результата.
5. Выведите на экран все промежуточные и итоговые результаты: исходное выражение, его ОПН-представление и финальный ответ. Вывод только финального числа без промежуточных результатов требование не выполняет.

Часть 5. Дополнительные задачи (необязательно)

- **Обработка ошибок:** Реализуйте проверку на корректность вводимого выражения (например, несбалансированные скобки,

некорректные операторы) и генерируйте информативные исключения.

- **Поддержка вещественных чисел** и унарного минуса (например, для выражений вида `-5 * (-3)`).
- **Расширение функционала:** Добавьте поддержку математических функций, таких как `sin`, `cos`, `sqrt`.

Критерии сдачи

Работа принимается только если одновременно выполнено всё:

- [ ] `Tokenize` корректно разбирает выражение без единого пробела (`"(3+4)*2"`), а не только пример из методички.
- [ ] Нигде в решении не используется `System.Collections.Generic.Stack<T>` — только собственный `MyStack` на `MyArrayList`.
- [ ] Часть 1 не реализована через `string.Split` / `Regex` по всей входной строке целиком.
- [ ] `ConvertToRpn` верно обрабатывает вложенные скобки и разный приоритет операторов (см. таблицу приоритетов в справочнике).
- [ ] `Calculate` даёт верный результат на выражениях минимум с 3 операциями и корректно обрабатывает деление на ноль и нехватку операндов исключением, а не крашем или тихим `NaN`.
- [ ] `Main` выводит все три значения: исходное выражение, его ОПН-представление и финальный результат.